

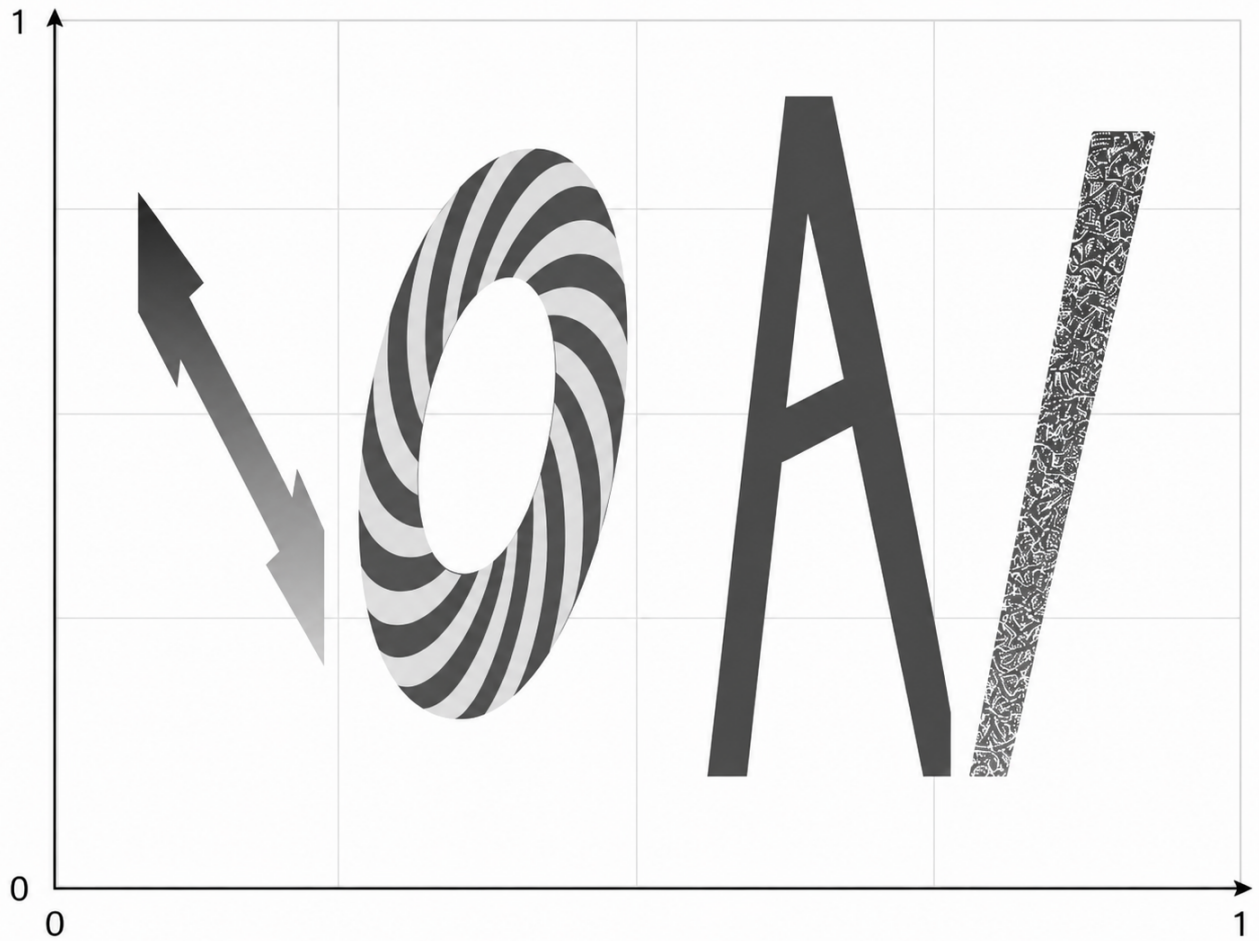
IOAI Field

- **Vaxt limiti:** 5 dəqiqə
- **Yaddaş:** 5 GB
- **Həllin ölçüsü:** `solution.ipynb`, `custom_model.py` birlikdə ≤ 1 MB
- **Əvvəlcədən təlimlənmiş modellər (pretrained models):** yoxdur — sıfırdan təlimləndirin, qiymətləndirmə zamanı internet yoxdur
- **Baza balı (Baseline Score):** 31.2187

Tapşırıq

Astana meri şəhəri stilizə edilmiş IOAI loqoları ilə bəzəmək istəyir. Bir statistik olaraq, o, hər şeyə — o cümlədən loqoya da — fəza funksiyası $F(x, y, \overline{W})$ kimi baxır; burada $x, y \in [0, 1]$ 2D müstəvidəki koordinatları, $\overline{W}$ isə hərflərin rəngləri və bucaqları kimi stilistik xüsusiyyətləri müəyyən edən gizli parametrlər dəstini təmsil edir.

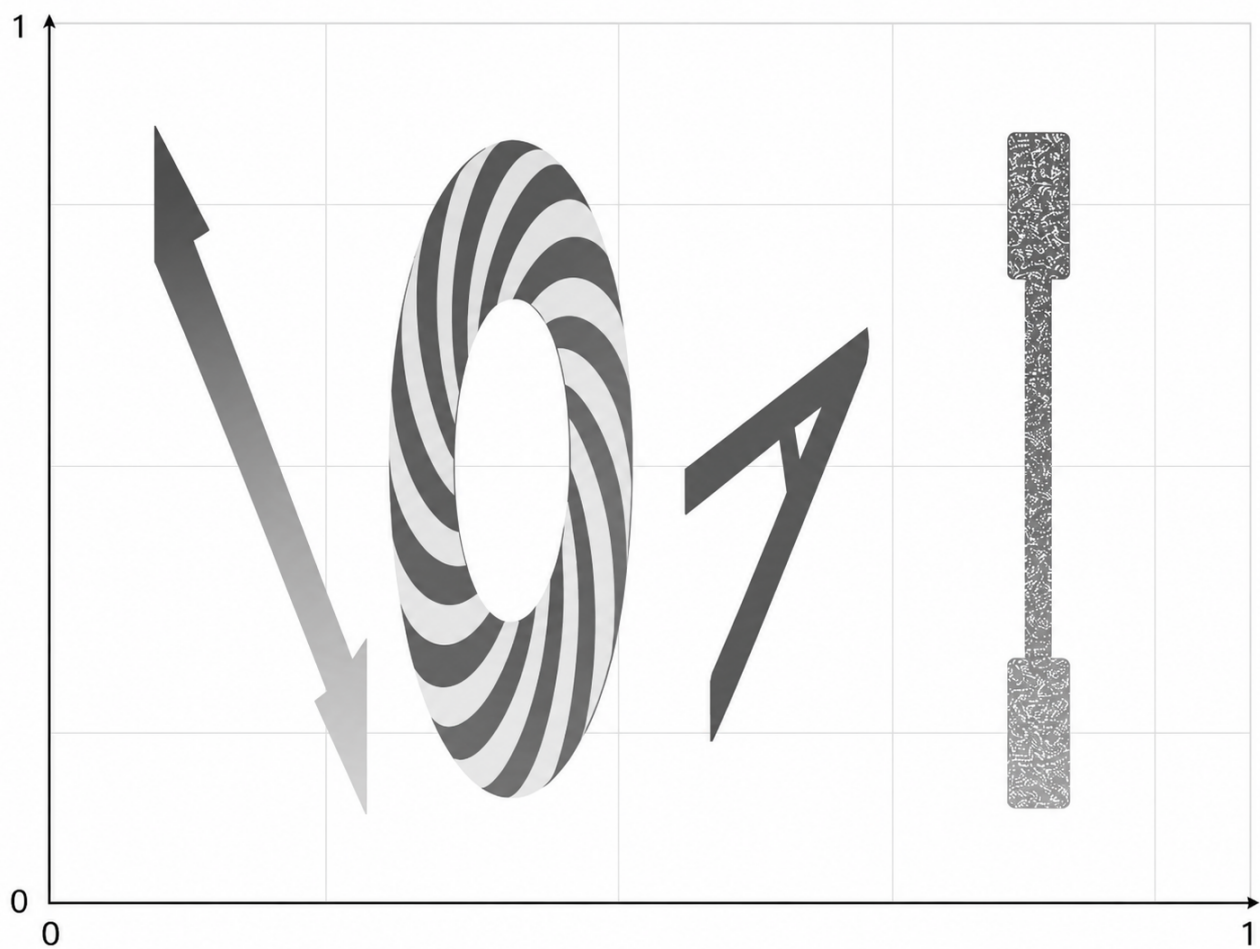
F aşkar riyazi tənlik kimi ifadə edilmək üçün çox mürəkkəb olduğundan, sizin tapşırığınız onu yaxınlaşdırmaq (approximate etmək) üçün bir neyron şəbəkəsi təlimləndirməkdir. Şəbəkə istənilən (x, y) koordinat cütü üçün bir **IOAI sahəsi** (IOAI field) dəyəri çıxaracaq və müstəvi boyunca loqonun tam istilik xəritəsi (heatmap) vizualizasiyasını yaradacaq. Aşağıda bəzi xüsusi gizli parametrlərə $\overline{W}$ malik F -in istilik xəritəsi vizualizasiyasının nümunəsi verilmişdir.

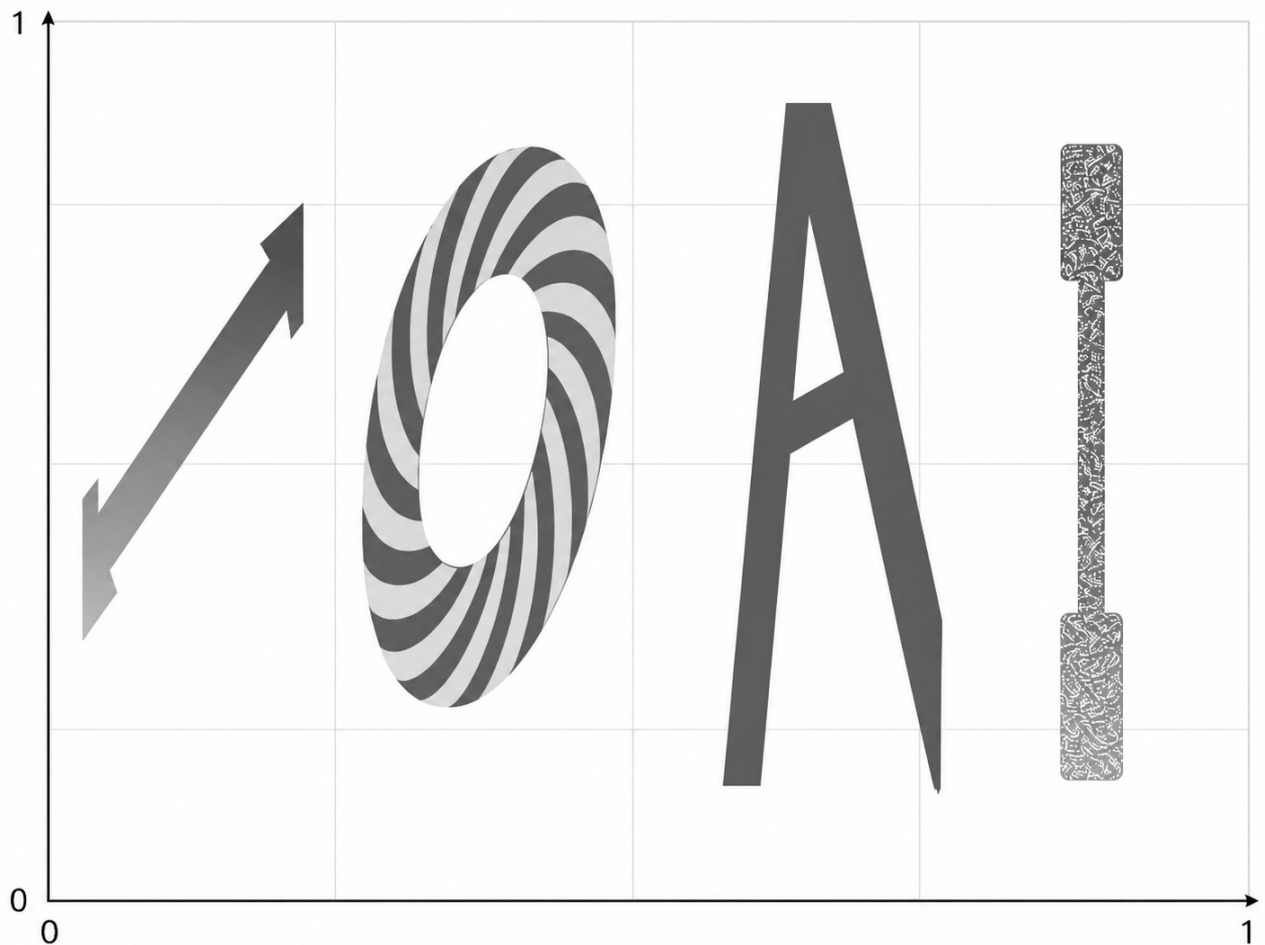


IOAI sahəsi nədən ibarətdir? Dörd hərf və fondan.

- Birinci **I** hərfinin daxilindəki dəyərlər xətti qradientlə çox böyükdür ($1e+10$ və daha çox)
- **O** hərfindəki dəyərlər spiral naxış nümayiş etdirir
- **A** hərfinin daxilindəki dəyər həmişə -1-dir
- Sonuncu **V** hərfinin daxilindəki dəyərlər, hətta eyni nöqtədə iki dəfə qiymətləndirilsə belə, $[-2026, 2026]$ diapazonundan olan təsadüfi dəyərlər olmalıdır
- Hərflərdən kənarda dəyər həmişə sıfırdır

Funksiya $\overline{W}$ gizli parametrlərinə malikdir ki, bu da hərflərin miqyasına və meyilliliyinə, həmçinin birinci **I** hərfinin daxilindəki dəyərlər diapazonuna təsir edir. Bununla belə, hərflər kəsişməyəcək. Aşağıda müxtəlif $\overline{W}$ ilə IOAI sahəsinin necə göründüyünə dair bir neçə əyani nümunə verilmişdir:





Sizə nə verilir:

Bu məsələdə HEÇ BİR məlumat dəsti (dataset) yoxdur. Əvəzində, sizə `data/train_config/field_config.json`-də yerləşən JSON konfiqurasiya faylı ilə tənzimlənən generator funksiyası verilir.

Test konfiqurasiyası gizlidir, lakin bənzər xarakter daşıyır. Sizin tapşırığınız istədiyiniz qədər məlumatdan istifadə edərək verilmiş generatora uyğunlaşmaqdır (fit etməkdir). Sizin "təlim" (train) və "test" paylanmalarınız eyni generatordan yaradılır - sadəcə hansı (x_i, y_i) nöqtələrində qiymətləndiriləcəyinizi bilmirsiniz.

Təqdim etdiyiniz həll (submission) aşağıdakılardan ibarət olmalıdır:

- `custom_model.py` kimi saxlanılan təlim modeli sinfi. Bu model `torch.nn.Module` sinfindən irs almalı (inherit etməli) və yalnız `torch` idxallarından (imports) istifadə etməlidir. O, `solution.ipynb` noutbukunda istifadə edilən `CustomModel` sinfini ehtiva etməlidir.
- `model.pt` çəkirlərini istehsal edəcək `solution.ipynb` noutbuku

Qiymətləndirmə

Hər bir region üçün minimum bal 0, maksimum bal isə 1-dir. Yekun bal bütün beş region üzrə (hər hərflər üçün dörd və fon) ortalama alınır və 100-ə vurulur. **Parametr cəriməsi** mövcuddur:

Əgər modelinizin 20260-dan çox parametri varsa, bal yarıya bölünür.

Parametrlərin sayı `sum(p.numel() for p in model.parameters())` ilə ölçülür. Modelinizin PyTorch `nn.Dropout` modelin bir hissəsi olmaqla stoxastik rejimdə də işləməsini gözləyirik.

Standart regionlar üçün

Hər bir R regionu üçün (birinci $\mathbb{I}$ hərflər, O, A, Background), biz modeli həqiqi dəyərləri v_i və proqnozları $\hat{v}_i$ olan $N_R = 512$ test nöqtələrində (x_i, y_i) qiymətləndiririk. Əsas metrik olaraq normallaşdırılmış Orta Mütləq Xəta (Mean Absolute Error - MAE) istifadə edirik. MAE aşağıdakı kimi təyin olunur:

$$\text{MAE}(R) = \frac{1}{N_R} \sum_{i=1}^{N_R} |\hat{v}_i - v_i|.$$

Və normallaşdırma aşağıdakı kimi həyata keçirilir:

$$\text{score}(R) = 1 - \min\left(\frac{\text{MAE}(R)}{s_R}, 1\right),$$

burada $s_R > 0$ miqyas sabitidir.

Sonuncu $\mathbb{I}$ hərflər regionu üçün

Bu regionda **qiymətləndirmə zamanı dropout aktiv edilir**. Hər bir j test nöqtəsi üçün:

1. $\hat{v}_{j,1}, \dots, \hat{v}_{j,K}$ nəticələrini əldə etmək üçün modeli $K = 10$ dəfə icra edirik.
2. Əgər hər hansı bir çıxış $[-2026, 2026]$ diapazonundan kənardadırsa, onda $\text{pointScore}(j) = 0$.
3. Əks halda, K çıxışının standart meylini (standard deviation) σ_j hesablayın və onu bala çevirin:

$$\text{pointScore}(j) = \min\left(\frac{\sigma_j}{s_E}, 1\right),$$

burada $s_E > 0$ sabit miqyas sabitidir.

Region balı regiondakı bütün nöqtələr üzrə ortalama dəyərdir:

$$\text{score}(I_{last}) = \frac{1}{N_E} \sum_{j=1}^{N_E} \text{pointScore}(j),$$

burada $N_E = K * N_R$.

Sadə dildə desək, müxtəliflik nə qədər çox olarsa, bu region üçün balınız bir o qədər yüksək olar. **Siz PyTorch `rand*` və `_uniform` funksiyaları da daxil olmaqla, xalis şəkildə təsadüfilikdən istifadə edə bilməzsiniz; təsadüfilik aktiv edilmiş dropout ilə çıxarışdan (inference) gəlməlidir.**

Həlli necə təqdim etməli

1. `solution.ipynb` faylını açın və bütün xanaları (cells) icra edin.
2. `custom_model.py` faylindəki `CustomModel` modelini təkmilləşdirin.
3. Sonuncu xananızın modelinizi `model.pt` faylına saxladığından əmin olun.
4. JupyterLab Git səhifəsində (tab) `solution.ipynb` və `custom_model.py` fayllarını hazırlayın (stage), şərh yazın və commit edin, sonra push edin.
5. Yarışma (Contest) səhifəsinə qayıdın və **Submit** düyməsini sıxın. Təqdim etmə şərh (submit comment) əvvəlki addımdakı şərh ilə eyni olmalıdır.